

Answers 3.9.

Yaw Assensoh opoku

```
WITH top_countries AS (SELECT ctry.country
FROM customer cust
INNER JOIN address AS addr ON cust.address_id = addr.address_id
INNER JOIN city AS city ON addr.city_id = city.city_id
INNER JOIN country AS ctry ON city.country_id = ctry.country_id
GROUP BY ctry.country
ORDER BY COUNT (cust.customer_id) DESC
LIMIT 10),
top_cities AS (SELECT city.city
FROM customer cust
INNER JOIN address AS addr ON cust.address_id = addr.address_id
INNER JOIN city AS city ON addr.city_id = city.city_id
INNER JOIN country AS ctry ON city.country_id = ctry.country_id
WHERE ctry.country IN (SELECT country FROM top_countries)
GROUP BY city.city
ORDER BY COUNT (cust.customer_id) DESC
LIMIT 10),
customer_payments AS (SELECT cust.customer_id, SUM(pmt.amount) AS
total_amount_paid
FROM payment pmt
INNER JOIN customer AS cust ON pmt.customer_id = cust.customer_id
INNER JOIN address AS addr ON cust.address_id = addr.address_id
INNER JOIN city AS city ON addr.city_id = city.city_id
WHERE city.city IN (SELECT city FROM top_cities))
```

GROUP BY cust.customer_id

ORDER BY total_amount_paid DESC

LIMIT 5)

SELECT AVG(total_amount_paid) AS avg_amount_paid

FROM customer_payments;

The screenshot shows a PostgreSQL query editor with a query that identifies the top 10 countries and cities by customer count, then calculates the average payment amount for the top 5 customers based on these criteria.

```
1 WITH top_countries AS (SELECT ctry.country
2 FROM customer cust
3 INNER JOIN address AS addr ON cust.address_id = addr.address_id
4 INNER JOIN city AS city ON addr.city_id = city.city_id
5 INNER JOIN country AS ctry ON city.country_id = ctry.country_id
6 GROUP BY ctry.country
7 ORDER BY COUNT (cust.customer_id) DESC
8 LIMIT 10),
9 top_cities AS (SELECT city.city
10 FROM customer cust
11 INNER JOIN address AS addr ON cust.address_id = addr.address_id
12 INNER JOIN city AS city ON addr.city_id = city.city_id
13 INNER JOIN country AS ctry ON city.country_id = ctry.country_id
14 WHERE ctry.country IN (SELECT country FROM top_countries)
15 GROUP BY city.city
16 ORDER BY COUNT (cust.customer_id) DESC
17 LIMIT 10),
18 customer_payments AS (SELECT cust.customer_id, SUM(pmt.amount) AS total_amount_paid
19 FROM payment pmt
20 INNER JOIN customer AS cust ON pmt.customer_id = cust.customer_id
21 INNER JOIN address AS addr ON cust.address_id = addr.address_id
22 INNER JOIN city AS city ON addr.city_id = city.city_id
23 WHERE city.city IN (SELECT city FROM top_cities)
24 GROUP BY cust.customer_id
25 ORDER BY total_amount_paid DESC
26 LIMIT 5)
27 SELECT AVG(total_amount_paid) AS avg_amount_paid
28 FROM customer_payments;
```

The results pane shows a single row with the average payment amount.

avg_amount_paid
120.322000000000000000

Showing rows: 1 to 1 | Page No: 1 of 1

Total rows: 1 | Query complete 00:00:00.068 | LF | Ln 24, Col 21

```

WITH customer_payments AS (SELECT cust.customer_id, ctry.country, SUM(pmt.amount)
AS total_paid

FROM payment AS pmt

INNER JOIN customer AS cust ON pmt.customer_id = cust.customer_id

INNER JOIN address AS addr ON cust.address_id = addr.address_id

INNER JOIN city AS city ON addr.city_id = city.city_id

INNER JOIN country AS ctry ON city.country_id = ctry.country_id

GROUP BY cust.customer_id, ctry.country),

top_5_customers AS (SELECT customer_id, country

FROM customer_payments

ORDER BY total_paid DESC

LIMIT 5),

all_customers_with_country AS (SELECT cust.customer_id, ctry.country

FROM customer AS cust

INNER JOIN address AS addr ON cust.address_id = addr.address_id

INNER JOIN city AS city ON addr.city_id = city.city_id

INNER JOIN country AS ctry ON city.country_id = ctry.country_id)

SELECT ac.country,

COUNT(DISTINCT ac.customer_id) AS all_customer_count,

COUNT(DISTINCT t5.customer_id) AS top_customer_count

FROM all_customers_with_country AS ac

LEFT JOIN top_5_customers AS t5 ON ac.customer_id = t5.customer_id

GROUP BY ac.country

ORDER BY all_customer_count DESC;

```

Dashboard X Properties X Statistics X Dependencies X Dependents X Processes X SQL X Rockbuster/postgres@PostgreSQL 17*

Query Query History

```

1 WITH customer_payments AS (SELECT cust.customer_id, ctry.country, SUM(pmt.amount) AS total_paid
2 FROM payment AS pmt
3 INNER JOIN customer AS cust ON pmt.customer_id = cust.customer_id
4 INNER JOIN address AS addr ON cust.address_id = addr.address_id
5 INNER JOIN city AS city ON addr.city_id = city.city_id
6 INNER JOIN country AS ctry ON city.country_id = ctry.country_id
7 GROUP BY cust.customer_id, ctry.country),
8 top_5_customers AS (SELECT customer_id, country
9 FROM customer_payments
10 ORDER BY total_paid DESC
11 LIMIT 5),
12 all_customers_with_country AS (SELECT cust.customer_id, ctry.country
13 FROM customer AS cust
14 INNER JOIN address AS addr ON cust.address_id = addr.address_id
15 INNER JOIN city AS city ON addr.city_id = city.city_id
16 INNER JOIN country AS ctry ON city.country_id = ctry.country_id)
17 SELECT ac.country,
18 COUNT(DISTINCT ac.customer_id) AS all_customer_count,
19 COUNT(DISTINCT t5.customer_id) AS top_customer_count
20 FROM all_customers_with_country AS ac
21 LEFT JOIN top_5_customers AS t5 ON ac.customer_id = t5.customer_id
22 GROUP BY ac.country
23 ORDER BY all_customer_count DESC;
24

```

Data Output Messages Notifications

Showing rows: 1 to 108 Page No: 1 of 1

	country character varying (50)	all_customer_count bigint	top_customer_count bigint
1	India	60	0
2	China	53	0
3	United States	36	1
4	Japan	31	0
5	Mexico	30	0
6	Brazil	28	1
7	Russian Federation	28	0
8	Philippines	20	0
9	Turkey	15	0
10	Indonesia	14	0
11	Nigeria	13	0
12	Argentina	13	0

Total rows: 108 Query complete 00:00:00.102 LF Ln 20, Col 38

Write 2 to 3 sentences explaining how you approached this step, for example, what you did first, second, and so on.

I started by finding the top 10 countries with the most customers by joining the customer, address, city, country tables and counting customers in each country. Then, from those countries, I picked the top 10 cities with the highest customer counts using the same method. After identifying these cities, I looked at customers living there and calculated their total payments by joining with the payment table. I grouped the data by customer and picked the top 5 customers who paid the most. Finally, I compared the total number of customers in each country to how many of them were in that top-paying group. This showed which countries had the most customers and where the highest-paying customers lived and I used CTEs for each step to keep the query organized, readable, and accuracy.

Step 2

2a. CTEs make complex queries easier to read and maintain but can sometimes be less efficient if executed multiple times, while subqueries may perform better for simple, single-use operations since they can be optimized more aggressively by the database but in all I think subqueries are much faster and shorter.

	Subqueries	CTE
Query 1 cost	167.82	166.08
	Query complete 00:00:00.115	Query complete 00:00:00.52
Query 2 cost	1245.10	1411.33
	Query complete 00:00:00.97	Query complete 00:00:00.79

The difference between the 2 wasn't that much in terms of cost and run time. I was a bit surprised because I thought subqueries would outperform CTE's.